ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 13:

Aim

To implement and demonstrate the Minimax Algorithm for Gaming (using Tic-Tac-Toe as a foundational game tree model) in Python to enable an AI agent to make optimal decisions in zero-sum, perfect-information games.

Algorithm

1. Define Game Tree and Players: Set up the game board representation, maximizing player (AI attempting to maximize score), and minimizing player (opponent attempting to minimize score).
2. Evaluation Function: Assess terminal states of the board:
    - Returns $+10$ for a win state of the maximizing player.
    - Returns -10 for a win state of the minimizing player.
    - Returns 0 for a draw or non-terminal evaluating branch.
3. Minimax Recursive Exploration:
    - Base Case: If the game is won, lost, or tied, return the respective evaluation score.
    - Maximizing Turn: Iterate through all legal moves, simulate the move, recursively call Minimax for the opponent's turn, and pick the maximum value.
    - Minimizing Turn: Iterate through all legal moves, simulate the move, recursively call Minimax for the AI's turn, and pick the minimum value.
4. Optimal Move Selection: The AI evaluates all available child nodes from the current state and executes the move corresponding to the highest minimax evaluation score.

CODE:

```python
class MinimaxGaming:
    def __init__(self):
        self.board = [[' ' for _ in range(3)] for _ in range(3)]
        self.ai = 'X'
        self.opponent = 'O'

    def print_board(self):
        for row in self.board:
            print("|" + "|".join(row) + "|")
        print()

    def is_moves_left(self):
        return any(cell == ' ' for row in self.board for cell in row)

    def evaluate(self):
        for row in self.board:
            if row[0] == row[1] == row[2] and row[0] != ' ':
                return 10 if row[0] == self.ai else -10

        for col in range(3):
            if self.board[0][col] == self.board[1][col] == self.board[2][col] and self.board[0][col] != ' ':
                return 10 if self.board[0][col] == self.ai else -10

        if self.board[0][0] == self.board[1][1] == self.board[2][2] and self.board[0][0] != ' ':
            return 10 if self.board[0][0] == self.ai else -10

        if self.board[0][2] == self.board[1][1] == self.board[2][0] and self.board[0][2] != ' ':
            return 10 if self.board[0][2] == self.ai else -10

        return 0

    def minimax(self, depth, is_max):
        score = self.evaluate()

        if score == 10 or score == -10:
            return score
        if not self.is_moves_left():
            return 0

        if is_max:
            best = -float('inf')
            for i in range(3):
                for j in range(3):
                    if self.board[i][j] == ' ':
                        self.board[i][j] = self.ai
                        best = max(best, self.minimax(depth + 1, False))
                        self.board[i][j] = ' '
            return best
        else:
            best = float('inf')
            for i in range(3):
```

```
Initial Game Board State:
|X|O|X|
|O|O| |
| | |X|

Optimal AI Move selected by Minimax: Row 1, Column 2
```

Result

The Minimax algorithm successfully evaluated the game tree from the given intermediate board configuration. By alternating between maximizing and minimizing future game trajectories, the AI computed the precise coordinate choice that guarantees the highest possible payoff against an optimal opponent.

Conclusion

The program successfully demonstrated the application of the Minimax algorithm for decision-making in gaming environments. By exhaustively traversing state spaces in zero-sum games, the algorithm ensures that the AI agent plays rationally and never loses when given complete information.